动手 Workshop：Cars CRUD 应用的 CI 与 CD 应用

目标：使用 GitHub Actions 自动测试 FastAPI Cars 应用，并在测试通过后构建其 Docker 镜像并发布到 Docker Hub。

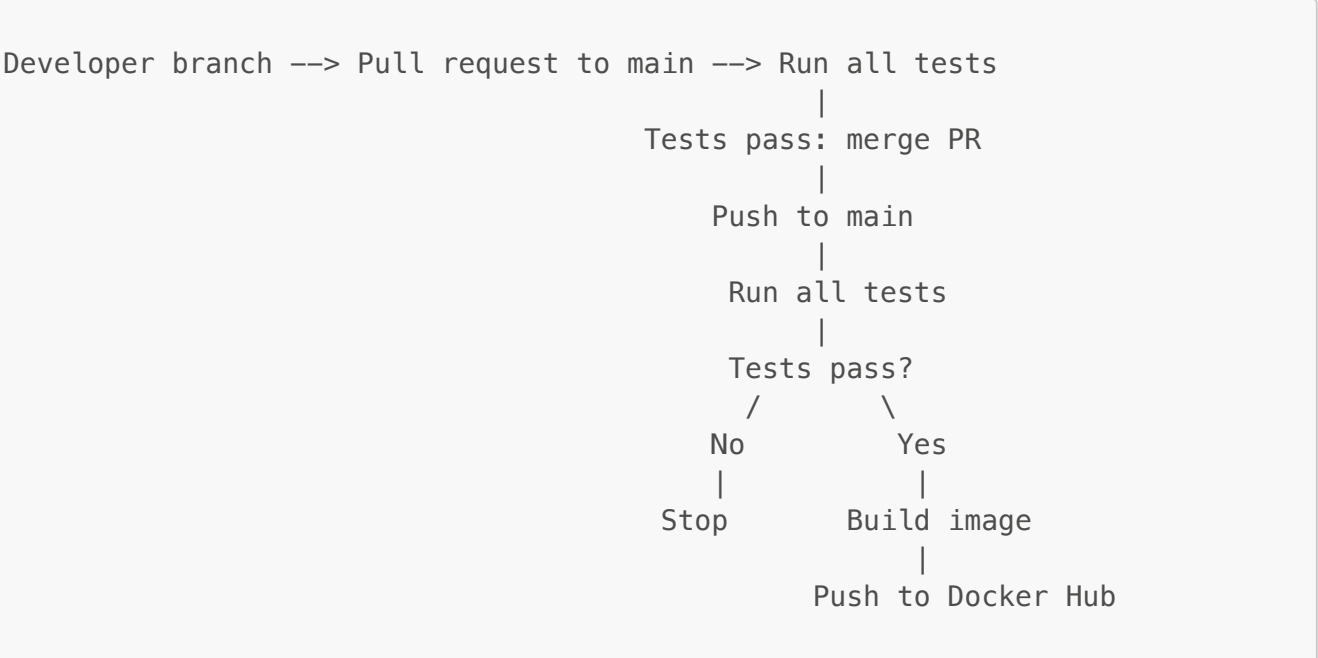
这是一个跟随练习。下面的工作流是完整的：复制、运行、检查结果，并尝试失败练习。

1. 我们要构建什么

我们的工作流有两个职责：

- 持续集成（CI）：运行单元、集成、API 和端到端（E2E）测试，每当有 pull request 以 main 为目标时，以及每当有提交推送到 main 时。
- 持续交付（CD）：在 main 上的测试通过后，构建 Cars CRUD Docker 镜像并发布到 Docker Hub。

在本 workshop 中，发布镜像就是我们的交付步骤。我们不在生产服务器上部署或启动应用。



事件	单元 + 集成 + API + E2E 测试	构建并发布 Docker 镜像
以 main 为目标的 pull request	是	否（发布作业被跳过）
推送或合并到 main	是	是，仅当所有测试通过
main 上的测试失败	失败	否

2. 检查你的项目

你的 Cars CRUD 项目应当已在 GitHub 上，包含这些文件：

```
cars-crud/
├── app/
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── api/
│   └── e2e/
├── Dockerfile
├── .dockerignore
├── requirements.txt
├── requirements-dev.txt
└── .github/
    └── workflows/
        └── tests.yml              # 来自之前的 GitHub Actions 练习
exercise
```

前提条件：你已完成之前的软件测试、Docker 和 GitHub Actions 练习。
你需要一个 GitHub 账户、一个 Docker Hub 账户，以及一个能构建应用的 Dockerfile。如果你用过之前的 MySQL/Compose 练习，可保留 compose.yaml；本工作流不需要它。

确认你的测试本地可运行

在项目根目录、Python 虚拟环境内：

```
python -m pip install -r requirements-dev.txt
python -m playwright install chromium
python -m pytest tests/unit tests/integration tests/api -q
python -m pytest tests/e2e -q
```

现有测试使用临时 SQLite 数据库，即使你也为 Docker Compose 配置了 MySQL。
E2E 测试 fixture 会自动启动和停止本地 FastAPI 服务器，因此 GitHub 工作流期间无需自己运行 Uvicorn。

创建工作分支

```
git switch main
git pull --ff-only origin main
git switch -c cars-ci-cd
```

如果你已有名为 cars-ci-cd 的分支，切换到它或选择另一个分支名。

3. 准备 Docker Hub

1. 登录 https://hub.docker.com/。
2. 在你自己的 Docker Hub 用户名下创建一个名为 cars-crud 的仓库。公开仓库对本练习最简单。
3. 打开 Docker 账户设置，找到 Personal access tokens，生成一个具有 Repo Read & Write 权限的令牌。给它一个可识别的描述，如 cars-github-actions。
4. 令牌出现时复制它。不要提交令牌、把它写进 YAML 文件或分享到截图中。

你发布的镜像名称类似：

```
YOUR_DOCKER_ID/cars-crud:latest
```

/cars-crud 之前的部分是你的 Docker Hub 用户名，不一定是 GitHub 用户名。

4. 将 Docker Hub 凭据保存为 GitHub Secrets

在 GitHub 仓库中（不是 Docker Hub）：

1. 打开 Settings > Secrets and variables > Actions。
2. 选择 New repository secret。
3. 使用下面的确切名称创建这两个 secret：

GitHub secret	值
DOCKERHUB_USERNAME	你的 Docker Hub 用户名
DOCKERHUB_TOKEN	Docker Hub 个人访问令牌

GitHub Actions 使用以下语法读取 secrets：

```
username: ${{ secrets.DOCKERHUB_USERNAME }}
password: ${{ secrets.DOCKERHUB_TOKEN }}
```

重要：令牌应放在 GitHub Secrets 中，绝不直接放进已提交的工作流文件。尽管用户名不敏感，但为保持练习一致，这里将两个值都存为 secrets。

检查点：你应在 GitHub 设置页看到两个 secret 名称。它们的值不会显示。

5. 编写 GitHub Actions 工作流

打开现有的 .github/workflows/tests.yml。用下面的工作流替换其内容。
除非你想让同样的测试运行两次，否则不要创建第二个测试工作流。

```
name: Cars CRUD - Test and Publish

on:
```

```yaml
  pull_request:
    branches: [main]
  push:
    branches: [main]

permissions:
  contents: read

jobs:
  tests:
    name: Run application tests
    runs-on: ubuntu-latest

    steps:
      - name: Check out repository
        uses: actions/checkout@v7

      - name: Set up Python
        uses: actions/setup-python@v7
        with:
          python-version: "3.12"

      - name: Install application and test dependencies
        run: |
          python -m pip install --upgrade pip
          python -m pip install -r requirements-dev.txt

      - name: Install Chromium for E2E tests
        run: python -m playwright install --with-deps chromium

      - name: Run unit tests
        run: python -m pytest tests/unit -q

      - name: Run integration tests
        run: python -m pytest tests/integration -q

      - name: Run API tests
        run: python -m pytest tests/api -q

      - name: Run E2E tests
        run: python -m pytest tests/e2e -q

  publish:
    name: Build and publish Docker image
    needs: tests
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest

    steps:
      - name: Check out repository
        uses: actions/checkout@v7

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v4
```

```
      - name: Log in to Docker Hub
        uses: docker/login-action@v4
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build and push Docker image
        uses: docker/build-push-action@v7
        with:
          context: .
          file: ./Dockerfile
          push: true
          tags: |
            ${{ secrets.DOCKERHUB_USERNAME }}/cars-crud:latest
            ${{ secrets.DOCKERHUB_USERNAME }}/cars-crud:sha-${{ github.sha
}}
```

理解重要行

工作流指令	含义
`on: pull_request`	当 PR 以 main 为目标或收到新提交时运行测试。
`on: push`	当变更到达 main 时也运行，包括合并 PR 之后。
`branches: [main]`	将这些事件限制在指定的目标/更新分支。
`permissions: contents: read`	只给工作流的 GitHub 令牌仓库读权限。
`jobs.tests`	运行四个测试组的作业。
`runs-on: ubuntu-latest`	使用 GitHub 托管的 Ubuntu 机器。
`actions/checkout`	将源代码下载到该机器上。
`actions/setup-python`	安装/选择 Python 3.12。
`requirements-dev.txt`	安装 FastAPI 和我们的 Python 测试依赖。
`playwright install --with-deps chromium`	安装 E2E 测试所需的浏览器和 Linux 库。
`needs: tests`	等待测试作业，且仅在其成功时运行。
`if: ... push ... main`	绝不从 pull request 发布；仅在推送到 main 后发布。
`docker/setup-buildx-action`	准备 Docker 的镜像构建器。
`docker/login-action`	使用存储的凭据向 Docker Hub 认证。

工作流指令	含义
`docker/build-push-action`	构建 Dockerfile，并因为 `push: true`.
`tags:`	发布 latest 和一个与 Git 提交 SHA 关联的标签。

第二个标签 sha-<commit SHA> 让你能精确识别哪个版本生成了镜像；
latest 指向最近发布成功的构建。

Docker Compose 和 MySQL 怎么办？

我们的测试针对隔离的 SQLite 文件运行，因此本 workshop 在 GitHub Actions 中不需要 MySQL 容器。
发布作业构建的是 FastAPI 应用镜像，而不是 Docker Compose
捆绑包。如果之后你想用 MySQL 运行，你的 Compose 配置还必须提供 MySQL
服务及其存储。

Azure Docker 镜像代理怎么办？

对中国的学生，本地拉取基础镜像通过课程的 Azure 代理可能更快：

```
# 标准基础镜像
# python:3.12-slim

# 可选：通过课程代理进行本地构建
docker build --build-arg
BASE_IMAGE=efreiwut.azurecr.io/cache/library/python:3.12-slim -t cars-
crud:local .
```

这使用了我们 Docker 指南中 Dockerfile 的 ARG BASE_IMAGE 设置。在 GitHub 托管的
运行器上，常规 Docker Hub 基础镜像（python:3.12-slim）在典型
情况下已足够，因此工作流无需代理构建参数。无论基础
镜像来自哪里，最终的应用镜像都会推送到你自己的 Docker Hub 账户。

6. 推送工作流并打开 pull request

从你的 cars-ci-cd 分支提交工作流：

```
git add .github/workflows/tests.yml
git commit -m "Test and publish Cars CRUD with GitHub Actions"
git push -u origin cars-ci-cd
```

如果你修改了 Dockerfile 或其他项目文件，也一并提交。

在 GitHub 中：

1. 打开 Pull requests > New pull request。

2. 设置 base: main 和 compare: cars-ci-cd。
3. 创建 PR 并打开 Checks（或从 Actions 标签页打开运行）。
4. 打开 Run application tests 作业并检查四个测试步骤。
5. 确认 Build and publish Docker image 作业显示 Skipped。这是正确的：PR 不应发布镜像。

检查点：PR 显示单元、集成、API 和 E2E 测试全部通过。Docker Hub 未从这次 PR 运行收到新镜像。

7. 证明失败的测试会停止工作流

合并之前，故意破坏你的一个现有测试：

1. Open `tests/unit/test_schemas.py` and find `test_color_is_optional`.
2. 暂时将：

```
assert car.color == ""
```

改为：

```
assert car.color == "Blue"
```

3. 在你的 PR 分支上提交并推送更改：

```
git add tests/unit/test_schemas.py
git commit -m "Demonstrate a failing unit test"
git push
```

4. 在 GitHub Actions 中打开最新的运行。单元测试步骤应当失败。后面的测试步骤可能因为执行在该失败处停止而被跳过。发布作业保持跳过。
5. 恢复 assert car.color == ""，然后提交并推送修复：

```
git add tests/unit/test_schemas.py
git commit -m "Restore passing unit test"
git push
```

6. 合并前确认最新的 PR 运行是绿色的。

为什么重要：失败的检查表明代码更改还没准备好合并。不要合并故意破坏的测试。

8. 合并到 main 并观察镜像发布

1. 将 pull request 合并到 main。
2. 打开 Actions 标签页，找到由推送到 main 触发的新运行。这与 PR 检查是不同的运行。
3. 观察 Run application tests 成功完成。
4. 观察测试成功后 Build and publish Docker image 开始。
5. 打开发布作业日志，确认 Log in to Docker Hub 和 Build and push Docker image 成功完成。
6. 打开你的 Docker Hub 仓库，确认存在两个标签：

```
YOUR_DOCKER_ID/cars-crud:latest
YOUR_DOCKER_ID/cars-crud:sha-<full Git commit SHA>
```

检查点：成功推送到 main 会产生一个新的 Docker Hub 镜像。失败的测试会停止发布作业。

9. 下载并运行已发布的镜像

在装有 Docker 的电脑上，将 YOUR_DOCKER_ID 替换为你的用户名：

```
docker pull YOUR_DOCKER_ID/cars-crud:latest

docker run -d \
  --name cars-from-hub \
  -p 127.0.0.1:8000:8000 \
  --mount type=volume,source=cars-published-data,target=/data \
  YOUR_DOCKER_ID/cars-crud:latest
```

打开 http://localhost:8000/ 或 http://localhost:8000/docs，然后创建一辆车。

- 该镜像包含 FastAPI 和原生的 HTML/CSS/JavaScript 前端。
- 命名卷持久化存储在 /data/cars.db 的默认 SQLite 数据库。
- 这是通过 Docker Hub 交付的构件进行的本地演示，不是生产部署。

如果 8000 端口已占用，先停止旧容器，或映射 127.0.0.1:8001:8000 并打开 http://localhost:8001/。清理时使用 docker stop cars-from-hub 后再 docker rm cars-from-hub（命名卷会保留，直到单独删除）。

10. 故障排查

症状	检查什么
PR 上没有出现工作流运行	确保文件是 .github/workflows/tests.yml，且 PR 的目标是 `main`.
`No module named pytest`	检查 requirements-dev.txt 的安装步骤。
Chromium 缺失或无法启动	Run `python -m playwright install --with-deps chromium` on 运行器。
E2E 服务器无法启动	检查 E2E fixture 的错误；它使用临时端口自动启动 FastAPI。
发布显示 Skipped（在 PR 上）	符合预期。发布仅限于推送到 main。
发布显示 Skipped on `main`	检查测试是否失败，并检查 if 条件。
Docker Hub 登录返回 `unauthorized`	Recheck `DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`, token validity, and 权限。
镜像推送返回 `denied`	确认 cars-crud 存在于正确的 Docker Hub 命名空间，且令牌有 Repo Read & Write 权限。
Docker 构建报 `Dockerfile` not found	确保 Dockerfile 在仓库根目录且已提交。
发布的镜像是旧的	检查最新的工作流运行，并将 sha-... 镜像标签与 main 上的提交比较。
本地网络访问 Docker Hub 较慢	如上所述，对受支持的基础镜像拉取使用 Azure 代理。它不是发布你的私有应用镜像的目的地。

11. 完成检查清单

- [ ] 我的应用有通过单元、集成、API 和 E2E 测试。
- [ ] 我的 Docker Hub 账户有 cars-crud 仓库。
- [ ] GitHub 仓库包含 DOCKERHUB_USERNAME 和 DOCKERHUB_TOKEN secrets。
- [ ] `.github/workflows/tests.yml` runs tests for PRs targeting `main`.
- [ ] 同一工作流在推送到 main 时再次运行测试。
- [ ] 发布作业依赖测试作业成功。
- [ ] 发布作业仅在推送到 main 时运行。
- [ ] 故意失败的测试让 PR 检查变红；恢复后检查变绿。
- [ ] 合并后，Docker Hub 包含新的 latest 和提交专属镜像标签。
- [ ] 我能在本地拉取并运行已发布的镜像。

提交：提供 GitHub pull request 链接、main 上成功的 GitHub Actions 运行链接和 Docker Hub 仓库链接。绝不要提交你的访问令牌。

官方参考

- GitHub Actions workflow events
- GitHub Actions repository secrets
- Docker Hub access tokens
- Docker build and push action
- Playwright Python on GitHub Actions